

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении практических работ
По дисциплине: «Функциональное программирование»
На тему: «Реализация проектов на языке Haskell»

Выполнил студент
группы 5130201/40002

_____ Семенов И. А.

Проверил
к.т.н

_____ Моторин Д. Е.

«_____» _____ 2026г.

Содержание

Введение	3
1 Временная диаграмма выполнения заданий	4
2 Структура и содержание разделов репозитория	5
3 Практическая работа №1	6
3.1 Выполненные задания	6
3.2 Использование нейросетей	6
3.3 Вопросы защиты и их решения	6
4 Практическая работа №2	7
4.1 Выполненные задания	7
4.2 Использование нейросетей	7
4.3 Вопросы защиты и их решения	7
5 Практическая работа №3	8
5.1 Выполненные задания	8
5.2 Использование нейросетей	8
5.3 Вопросы защиты и их решения	8
6 Практическая работа №4	9
6.1 Выполненные задания	9
6.2 Использование нейросетей	9
6.3 Вопросы защиты и их решения	9
7 Практическая работа №5	10
7.1 Выполненные задания	10
7.2 Использование нейросетей	10
7.3 Вопросы защиты и их решения	10
Заключение	11
Список литературы	11
Приложение	12

Введение

В рамках дисциплины «Функциональное программирование» были выполнены практические работы, направленные на освоение языка Haskell и основных функциональных абстракций. В работах использовались рекурсивные функции, алгебраические типы данных, классы типов, парсеры, монады, монада *RWS*, трансформеры монад, собственный трансформер *MyStateT*, а также тестирование свойств с помощью QuickCheck.

Отчёт оформлен по структуре ранее подготовленного отчёта: содержит постановку задачи, аналитическую часть, проектирование, программную реализацию, историю коммитов и заключение. Отдельно приведены временная диаграмма коммитов и сдач практических работ, структура репозитория, ответы на вопросы защиты и сведения об использовании нейросетей.

1 Временная диаграмма выполнения заданий

На рисунке 1 представлена временная диаграмма выполнения практических работ. Синими точками отмечены дни, в которые выполнялись коммиты в репозиторий; число над точкой показывает количество коммитов за день. Красными точками отмечены даты сдачи или состояние защиты практических работ.



Рис. 1: Временная диаграмма коммитов и сдач практических работ

2 Структура и содержание разделов репозитория

Репозиторий с практическими работами имеет следующую структуру:

```
practice/
|-- practice_01/
|-- practice_02/
|   |-- MyEvolProject/
|   \-- myParser/
|-- practice_03/
|   |-- glitcher/
|   \-- maze-rws/
|-- practice_04/
|   \-- mazeGame/
|       |-- app/
|       |-- data/
|       \-- src/
|-- practice_05/
|   \-- qc/
|       |-- app/
|       |-- src/
|       \-- test/
\-- report/
    |-- main.tex
    |-- main.pdf
    \-- tex/
```

Каталог `practice_01` содержит первые функции и решения на защиту. В `practice_02` находятся проекты с пользовательскими типами и парсерами. В `practice_03` расположены проекты `glitcher` и `maze-rws`. В `practice_04/mazeGame` находится текстовая игра по лабиринту с собственным трансформером состояния. В `practice_05/qc` находится проект с QuickCheck-тестами. Все изменения фиксировались в системе контроля версий Git.

3 Практическая работа №1

3.1 Выполненные задания

В первой практической работе были реализованы функции для работы со списками, кортежами, свёртками и типом *Maybe*. В отдельной части были добавлены функции *myZipSave*, *myUnzipSave*, *myReverse*, *myFoldl1*, *myFoldr1*, *myTakeWhile*, *mySpan*, *myMaybe*, а также функции для пользовательского списка и примеры из лекции про приготовление тортов.

Основные коммиты практики: f0752bd, 5baf79c, 78c0460, f476e64, c575c1c, 1c028cd, 5b5342c.

3.2 Использование нейросетей

Использование нейросети было зафиксировано в комментариях к файлам практики. Модель использовалась для обсуждения структуры функций и проверки отдельных реализаций, после чего код приводился к стилю практической работы.

3.3 Вопросы защиты и их решения

На защите была получена задача реализовать функцию типа *Either* $(a \rightarrow a) (c \rightarrow c) \rightarrow [a] \rightarrow [c] \rightarrow [(a, c)]$. Если передан *Left* *f*, функция применяется к элементу первого списка; если *Right* *f* — к элементу второго списка.

Листинг 1: Решение задачи защиты практической работы №1

```
foo :: Either (a -> a) (c -> c) -> [a] -> [c] -> [(a, c)]
foo _ [] _ = []
foo _ _ [] = []
foo (Left f) (x : xs) (y : ys) = (f x, y) : foo (Left f) xs ys
foo (Right f) (x : xs) (y : ys) = (x, f y) : foo (Right f) xs ys
```

4 Практическая работа №2

4.1 Выполненные задания

Во второй практической работе были созданы проекты `MyEvolProject` и `myParser`. В первом проекте реализован тип `MyEvolution` и представители классов типов `Show`, `Read`, `Eq`, `Ord`, `Enum`, `Bounded`. Также добавлен вариант типа с автоматическим выводом представителей классов типов.

Во втором проекте реализованы пользовательские типы `MyEither`, `MyMaybe`, `MyTree` и представители классов типов для них. Отдельно были реализованы парсеры: лекционный парсер, парсер на `Parsec` и парсер на `Attoparsec`.

Основные коммиты практики: 49f3596, bd2c465, 75ef59e, 17cca30, 0f6b475, fcf49d8, b78507d, 5c30b85, dc5e0eb.

4.2 Использование нейросетей

Сведения об использовании модели были добавлены в README и комментарии к файлам практики. Нейросеть использовалась для пояснения представителей классов типов и для проверки структуры парсеров.

4.3 Вопросы защиты и их решения

На защите разбиралось выражение с applicative-оператором (`<*>`) для типа `(Either a Int, String)`. Для пары applicative-операции выполняются покомпонентно: в первой компоненте работает `Either`, во второй компоненте строки объединяются как моноид.

Листинг 2: Выражение для защиты практической работы №2

```
( Right (\x y -> x + y + 1 :: Int)
  <*> Right (1 :: Int)
  <*> Right (1 :: Int)
  , "abc"
) :: (Either a Int, String)
```

Результат выражения равен `(Right 3, "abc")`.

5 Практическая работа №3

5.1 Выполненные задания

Практическая работа состояла из двух проектов. В проекте `glitcher` была реализована обработка изображения: случайная замена байта, сортировка участка байтов и последовательное применение нескольких действий с сохранением промежуточных файлов. Действия были перенесены в `main`, чтобы точка входа явно читала файл, применяла операции и записывала результаты.

В проекте `maze-rws` был реализован лабиринт и поиск пути. Лабиринт представлен как граф комнат, а поиск выполняется обходом в глубину. Основная функция обхода — `tryRooms`: она перебирает соседние комнаты, рекурсивно углубляется в непосещённые варианты и возвращает найденный путь.

Основные коммиты практики: `d04a59e`, `b3058b5`, `f94caad`, `5f8aa15`, `675192e`, `aac32e4`, `832e62f`, `b766d6e`, `5d15e30`, `88cc5cd`.

5.2 Использование нейросетей

В проекте `glitcher` был добавлен файл с историей запросов к LLM. Модель использовалась для обсуждения структуры проекта, разбиения чистых и *IO*-функций и переноса действий в `main`.

5.3 Вопросы защиты и их решения

На защите обсуждалось, где используется монада *RWS* и почему функция лабиринта имеет такой тип. *Reader* хранит неизменяемое окружение, то есть карту лабиринта; *Writer* накапливает лог обхода; *State* хранит состояние обхода. Такой тип нужен, потому что поиск одновременно читает лабиринт, меняет состояние и записывает сообщения.

Также обсуждалось, где именно находится обход в глубину. В данной реализации он находится в функции `tryRooms`: функция берёт список возможных комнат и пробует их по одной, углубляясь рекурсивно до тех пор, пока не найдёт целевую комнату или не переберёт все варианты.

6 Практическая работа №4

6.1 Выполненные задания

В четвёртой практической работе был создан Stack-проект `mazeGame`. Сразу после инициализации был удалён стандартный файл `Lib`. В проект добавлены зависимости, файл с лабиринтом главного здания, функция чтения файла в формат `IO [(String, String)]`, собственный трансформер `MyStateT` и функция запуска игры.

Лабиринт представлен типом `Map Room [Room]`, где ключ — комната, а значение — список соседних комнат. Функция `addRoomPair` добавляет связь между двумя комнатами в обе стороны, поэтому из списка пар строится неориентированный граф.

Тип игрового действия после рефакторинга:

Листинг 3: Стек трансформеров игры

```
type Game = ReaderT Maze (WriterT [String] (MyStateT Room IO))
```

Основные коммиты практики: `e4678a9`, `94e6e47`, `1f73730`, `bba7573`, `95f97e2`, `cbaa959`, `99277ac`, `515420e`.

6.2 Использование нейросетей

Для четвёртой практики был создан файл с перепиской с LLM. После завершения работы лог больше не дополнялся. Модель использовалась для обсуждения трансформера, порядка терминирования и объяснения функций `lift`, `ask`, `getS`, `runGame`.

6.3 Вопросы защиты и их решения

Трансформер `MyStateT` объявлен в отдельном файле `MyStateT.hs`. Он является конструктором, который принимает монаду и возвращает новую монаду с добавленным состоянием и сохранённым функционалом внутренней монады.

Терминирование трансформеров происходит в функции `runGame`. Каждый слой раскрывается своей функцией `runReaderT`, `runWriterT`, `runMyStateT`; после раскрытия всех слоёв остаётся терминатор `IO`.

Листинг 4: Запуск игры

```
runMyStateT (runWriterT (runReaderT game (buildMaze roomPairs))) "start"
```

В этом выражении `game` передаётся в `runReaderT`, туда же передаётся окружение — `buildMaze roomPairs`. После этого раскрывается `WriterT`, затем `MyStateT` получает начальное состояние `"start"`. `IO` вызывается не самим написанием выражения, а при выполнении полученного `IO`-действия, например через `bind` или в `main`.

7 Практическая работа №5

7.1 Выполненные задания

В пятой практической работе был создан Stack-проект `qc`. В проект добавлена библиотека `QuickCheck` и реализованы функции `addMod` и `reverseWords`. Для тестов создан каталог `test` и файл `Spec.hs`.

Перед свойствами были написаны генераторы:

Листинг 5: Генераторы для тестов

```
genWord :: Gen String
genWord = listOf1 $ elements ['a' .. 'z']

genSentence :: Gen String
genSentence =
  choose (0, 8) >>= \n ->
    replicateM n genWord >>= \wordsList ->
      return $ intercalate " " wordsList

genMod :: Gen Int
genMod = choose (1, 1000)
```

Всего написано семь свойств: три для `addMod` и четыре для `reverseWords`. Тесты запускаются с подписями в консоли.

Основные коммиты практики: 7bf6419, ad439ae, 928bdbe, 462c309, c7d26e5, 707ccf8.

7.2 Использование нейросетей

История взаимодействия с LLM для пятой практики сохранена в файле `practice_05/qc/llm.txt`. Модель использовалась для обсуждения генераторов, свойств `QuickCheck`, типа `Arbitrary` и формы записи тестов.

7.3 Вопросы защиты и их решения

`Arbitrary` — это класс типов `QuickCheck` для значений, которые можно автоматически генерировать. Если свойство имеет тип вроде `Int -> Int -> Bool`, `QuickCheck` берёт генераторы из представителей `Arbitrary` для `Int`. В работе также использовались собственные генераторы, чтобы ограничить модуль положительными значениями от 1 до 1000 и генерировать осмысленные строки из слов.

Функция `unwords` нужна для обратной сборки предложения из списка слов. `words` разбивает строку на слова, `reverse` меняет порядок слов, а `unwords` соединяет их обратно через пробел.

Заключение

В ходе выполнения практических работ были освоены основные средства функционального программирования на языке Haskell: рекурсия, сопоставление с образцом, алгебраические типы данных, классы типов, монады, трансформеры монад и тестирование свойств.

В первой работе были реализованы базовые функции и решение задачи защиты с типом *Either*. Во второй работе были реализованы пользовательские типы, представители классов типов и парсеры. В третьей работе были созданы проекты для глитч-обработки изображения и поиска в лабиринте с использованием *RWS*. В четвёртой работе был реализован Stack-проект текстовой игры по лабиринту с собственным трансформером *MyStateT*. В пятой работе были реализованы функции и проверены их свойства с помощью QuickCheck.

Все основные этапы работы фиксировались в Git. Для отчёта была построена временная диаграмма коммитов и сдач, приведена структура репозитория, перечислены ключевые коммиты и отражено использование нейросетей в файлах практик.

Список литературы

- [1] Haskell 2010 Language Report [Электронный ресурс]. URL: <https://www.haskell.org/onlinereport/haskell2010/> (дата обращения: 25.05.2026).
- [2] The Haskell Tool Stack [Электронный ресурс]. URL: <https://docs.haskellstack.org/> (дата обращения: 25.05.2026).
- [3] QuickCheck: Automatic testing of Haskell programs [Электронный ресурс]. URL: <https://hackage.haskell.org/package/QuickCheck> (дата обращения: 25.05.2026).
- [4] Monad transformer library [Электронный ресурс]. URL: <https://hackage.haskell.org/package/mtl> (дата обращения: 25.05.2026).
- [5] Материалы лекций и практических занятий по дисциплине «Функциональное программирование», СПбПУ, 2026.

Приложение

Ссылка на репозиторий GitHub:

<https://github.com/silog/SemenovIAFP>

Листинг П.1 — фрагмент истории коммитов

```
515420e 2026-05-23 Рефакторинг: меняет порядок трансформеров в типе Game
99277ac 2026-05-22 Revert: убран переход назад из 4 практики
88cc5cd 2026-05-22 Рефакторинг: поправлен вывод в файлов в 3 практике
707ccf8 2026-05-22 Рефакторинг: добавлены комментарии к тестам
c7d26e5 2026-05-22 Рефакторинг: тесты переписаны через forAll
462c309 2026-05-22 Рефакторинг: обновлены QuickCheck-тесты и добавлена и
928bdbe 2026-05-22 Добавлены QuickCheck-тесты для функций 5 практики
ad439ae 2026-05-22 Доавлены функции addMod и reverseWords
7bf6419 2026-05-22 Инициализирован stack-проект qc в практике №5
95f97e2 2026-05-22 Добавлены функции runGame и gameLoop
bba7573 2026-05-21 Добавлен трансформер MyStateT
1f73730 2026-05-21 Добавлена функция чтения лабиринта из файла
94e6e47 2026-05-21 Добавлен лабиринт главного здания для 4 практики
e4678a9 2026-05-21 Инициализирован stack-проект mazeGame
b766d6e 2026-05-15 Добавлены функции поиска в лабиринте
aac32e4 2026-05-15 Инициализирован stack-проект с лабиринтом
d04a59e 2026-05-08 Инициализирован stack проект glitcher
dc5e0eb 2026-04-24 Добавлены парсеры, реализованные с использованием биб.
17cca30 2026-04-23 Инициализирован stack-проект MyParser
49f3596 2026-04-20 Добавлен файл MyEvolModule
5b5342c 2026-03-20 Ответ на вопрос на защите первой лабораторной работы
f0752bd 2026-03-05 Добавлены функции myZipSave
```